

INFORME FINAL DE PROYECTO SISTEMA DE GESTIÓN DE TORNEOS

Asignatura: Desarrollo Orientado a Objetos (DOO)

Integrantes: Esteban Andrés Astete Cifuentes, Joel Tomás Andrés Rojas Seals
Benjamín Alexander Belmar Araus

Profesor: Geoffrey Hecht

Julio 2026

1. Decisiones de Diseño y Arquitectura

Para el desarrollo del proyecto, decidimos separar la lógica de negocio (Modelo) de la interfaz gráfica (UI). Elegimos esta arquitectura porque el sistema tenía que ser escalable, permitiendo que se puedan agregar otros deportes o torneos sin tener que cambiar el código base, esto nos permite cumplir con el principio Abierto/Cerrado de SOLID, resultando un sistema flexible y fácil de extender.

2. Patrones de Diseño Utilizados

Elegimos 3 de patrones de diseño para resolver problemas de acoplamiento y flexibilidad:

A. Patrón Strategy (Estrategia)

- **¿Por qué se eligió?** El sistema administra varios deportes (Fútbol, Tenis, Ajedrez) y formatos (Liga, Partido Único), cada uno con reglas de puntos y emparejamiento diferentes. Llenar la clase Torneo de condicionales if-else no hubiese cumplido los principios SOLID.
- **Implementación:**
 - Se creó la interfaz Disciplina que define métodos como `puntajeVictoria()` y `validarMatchResultado()`. Las clases (Futbol, Tenis, Basket, etc.) tienen esta interfaz.
 - Creamos la interfaz TorneoFormato con métodos como `generarMatches()`. Las clases Liga y PartidoUnico tienen su propio algoritmo.
 - Finalmente usamos también la UI con la interfaz Rol para separar los permisos y vistas entre `OrganizadorRol` y `EspectadorRol`.

B. Patrón Factory Method (Fábrica)

- **¿Por qué se eligió?** Teníamos que encontrar una forma de instanciar las disciplinas y formatos a partir de lo que elija el usuario en la interfaz gráfica, el objetivo es evitar que el código de la interfaz estuviese lleno de condicionales.
- **Implementación:** Utilizamos como punto de partida los enumeradores `DisciplinaEnum` y `FormatoEnum`. Cada elemento del Enum implementa un método abstracto (ej. `crearDisciplina()`), actuando como una fábrica que devuelve la instancia (`new Futbol()`, `new Liga()`).

C. Patrón Observer (Observador)

- **¿Por qué se eligió?** La interfaz gráfica (el Bracket visual) tiene que actualizarse de forma automática cuando termina un partido, pero sin que la lógica de la clase BracketsGestion tuviera nada que ver con Swing.
- **Implementación:** Creamos la interfaz EstadoBracketsGestion de forma que la clase BracketsGestion actúa como el *Sujeto* que mantiene una lista de observadores y llama a `notificarMatchActualizado()`. El panel visual BracketVer implementa la interfaz y se suscribe para redibujar la pantalla gráficamente mediante `SwingUtilities.invokeLater()` cuando los datos cambian.

3. Problemas Encontrados (Desafíos Técnicos)

Durante el desarrollo, tuvimos los siguientes desafíos de implementación:

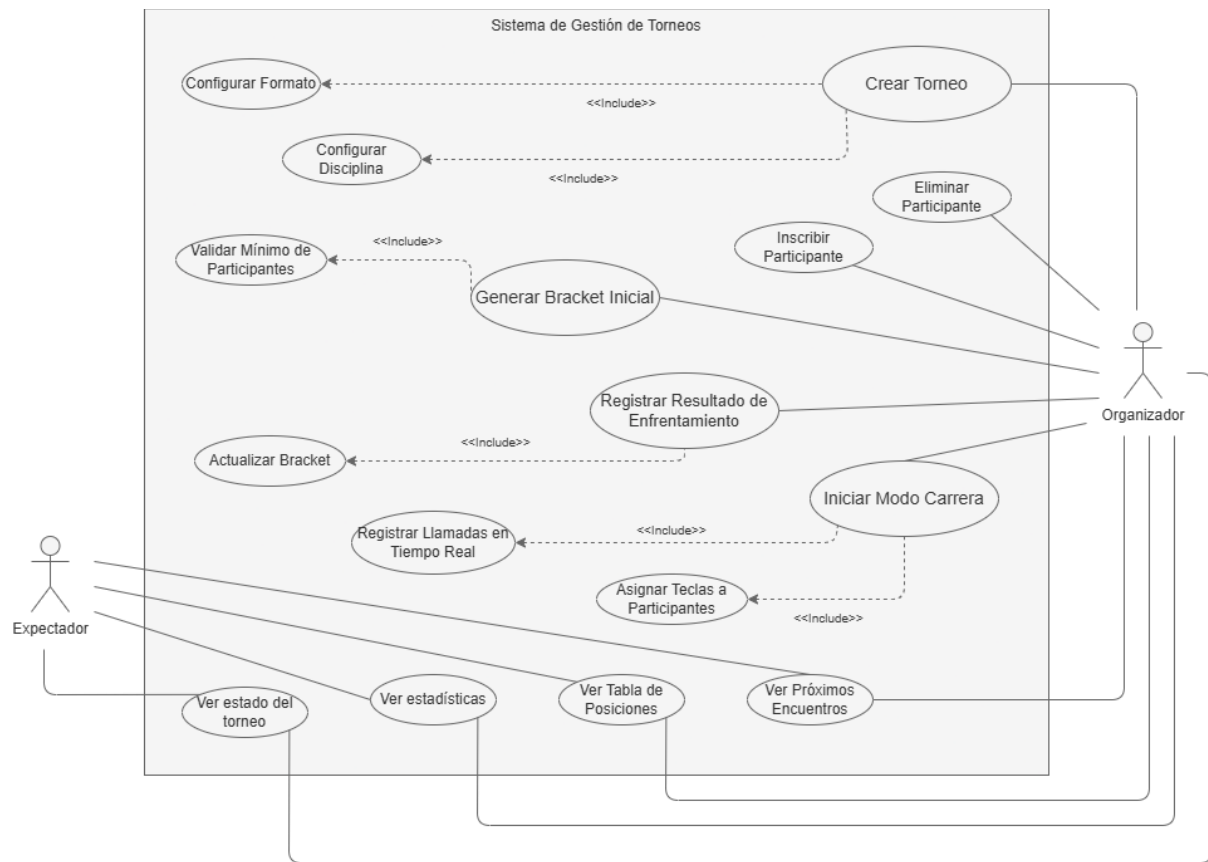
1. **Algoritmo de Brackets asimétricos (Byes):** Al generar el formato de eliminación directa en la clase PartidoUnico, las llaves solo funcionaban con 4, 8, 16 participantes (potencias de 2), con 6 o 10 participantes el sistema fallaba, ¿Solución? diseñar un algoritmo que rellena espacios vacios con participantes nulos ("Byes") así el torneo tiene una base logica aunque falten participantes.
2. **Manejo del historial para Deshacer (Undo):** En el diálogo de registro de eventos (RegistrarResultadoDialogo), anular un gol o un punto era difícil porque la clase MatchResultado no entregaba métodos para eliminar sucesos de forma segura, no se podía eliminar el ultimo movimiento. La solución fue crear un historial en la UI que, al deshacer, vuelve a calcular el resultado sin ver el ultimo cambio, en vez de editar el objeto original como estaba antes.
3. **Concurrencia en la Interfaz Gráfica:** la interfaz se congelaba por un momento al actualizar el marcador, el modelo intentaba cambiar la UI desde un hilo secundario. ¿Solución? implementar `SwingUtilities.invokeLater()` en BracketVer esto permitio sincronizar el dibujo en la pantalla con el hilo principal de Java, para evitar congelamientos de pantalla.

4. Propuesta de Mejoras Futuras

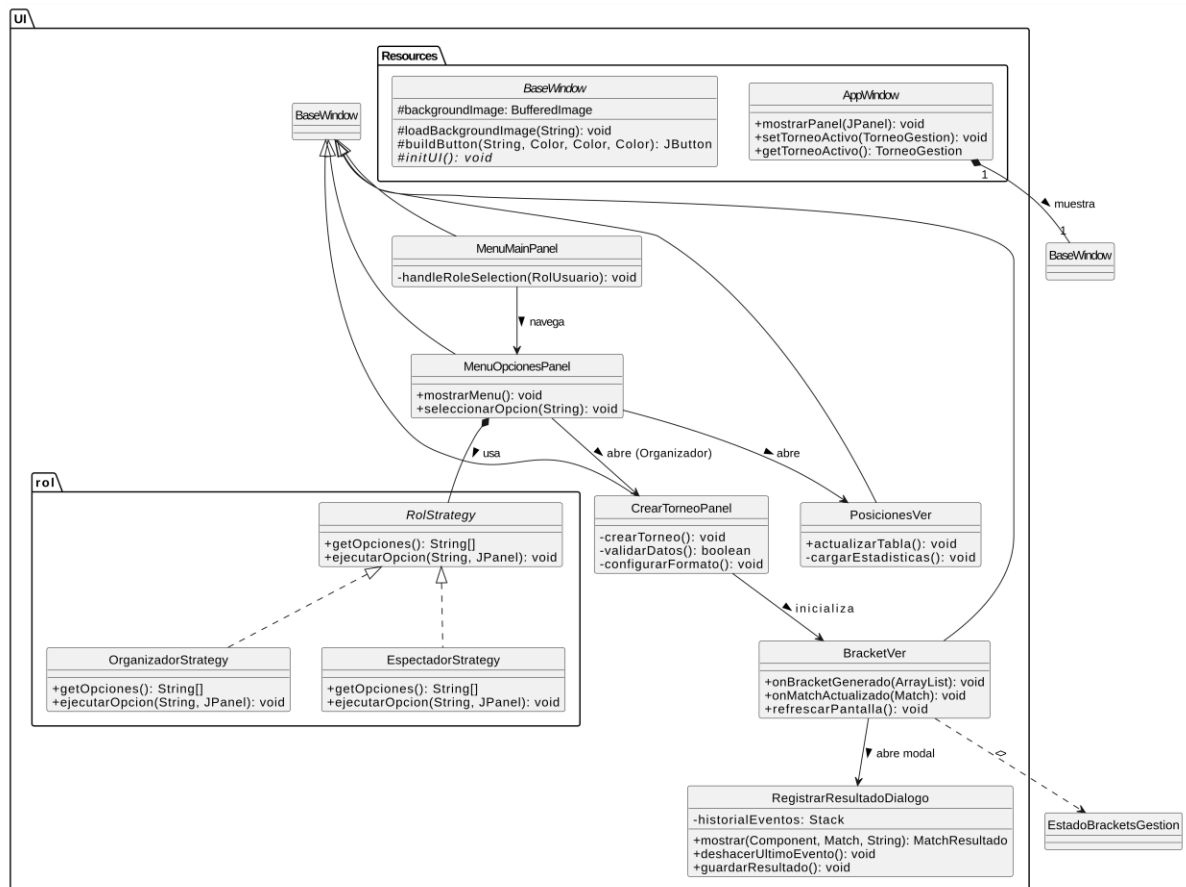
Si se dispusiera de más tiempo para iterar sobre el software, proponemos las siguientes mejoras:

- **Completar la lógica de "Losers Bracket":** Actualmente, la clase *IdaVuelta* necesita expandir su implementación algorítmica para manejar el flujo de perdedores de forma automática.
- **usabilidad (Drag & Drop):** Actualmente, los emparejamientos en los brackets se hacen de forma automática. Como mejora de uso, proponemos implementar una función de *Drag & Drop* (arrastrar y soltar) usando los eventos de ratón (*MouseListener*) de Java Swing. Esto le permitiría al organizador mover manualmente los botones de los equipos en la pantalla para configurar las llaves iniciales a su gusto antes de arrancar el torneo.
- **Vistas de Estadísticas:** Concluir el desarrollo visual de la tabla de posiciones (*PosicionesVer*) para los formatos de Liga, permitiendo ordenar a los equipos por diferencia de goles o sets según el deporte seleccionado.

5. Diagrama de Casos



6. Diagrama UML



7. Diagrama UML Lógica

